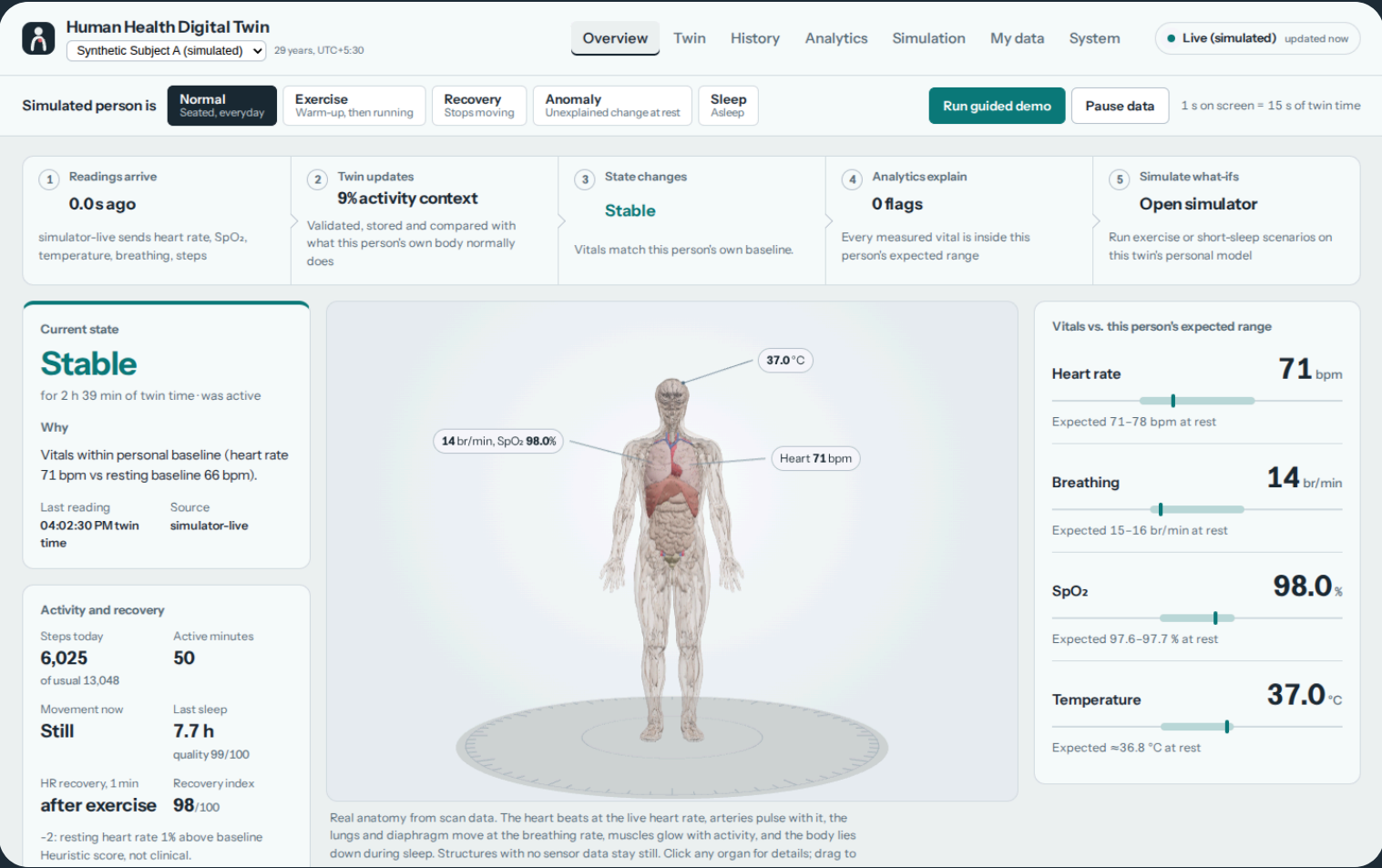


Human Health Digital Twin

A live model of one person that learns their normal, explains what changed, and simulates what-ifs.

[Team name] · [Member names]
[College name]



THE PROBLEM

One threshold for everyone fails both ways

150 bpm

while running

Threshold app: alarm (false)

Personal twin: expected for this effort

85 bpm

sitting still, normal is 66

Threshold app: silence (missed)

Personal twin: 29% above their own rest

OUR APPROACH

A live model, not just a 3D body

01

Data

Simulator, wearable exports or your
Android phone

02

Twin state

Personal baseline, context, state machine

03

Analytics

Flags with reasons, trends, wellness

04

Visualisation

Scan-based 3D anatomy driven by the
data

05

Simulation

What-if runs on this person's model

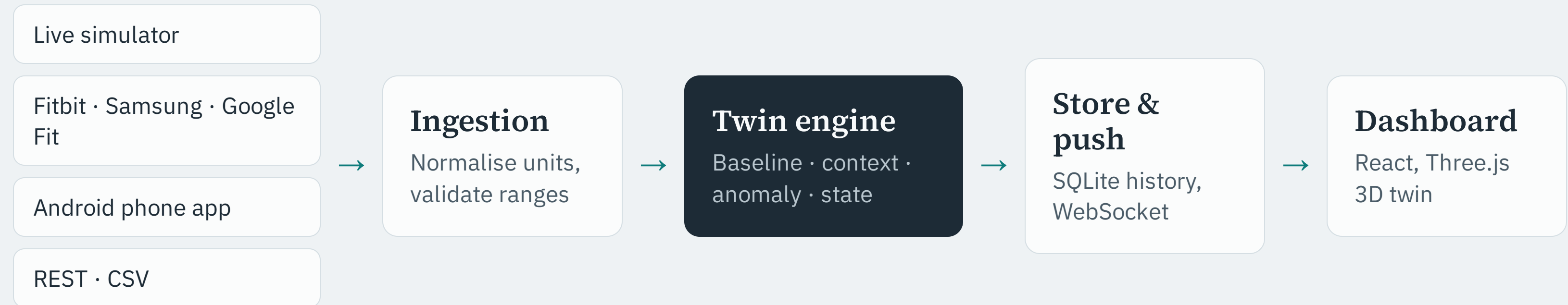
06

Feedback

Every reading re-syncs the twin

ARCHITECTURE

Any source, one engine



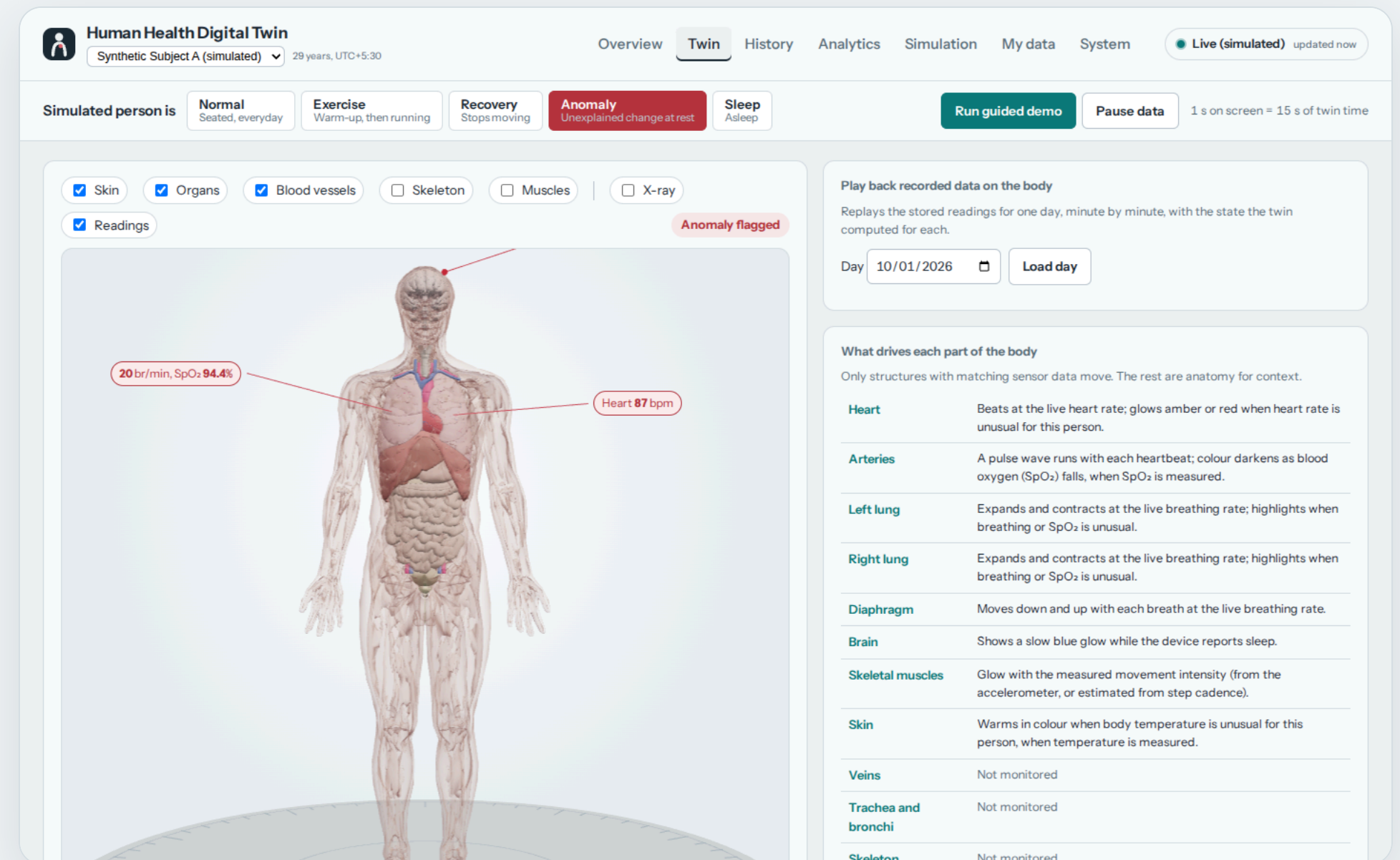
The same engine runs live data, history backfill, real uploads, phone sync, the what-if simulator and the validation harness.

VISUALISATION

Real anatomy, driven by data

- Heart beats at the live heart rate
- Arteries pulse and darken as SpO₂ falls
- Lungs and diaphragm at the breathing rate
- Muscles glow with activity
- Body lies down when asleep
- Unmonitored organs never move

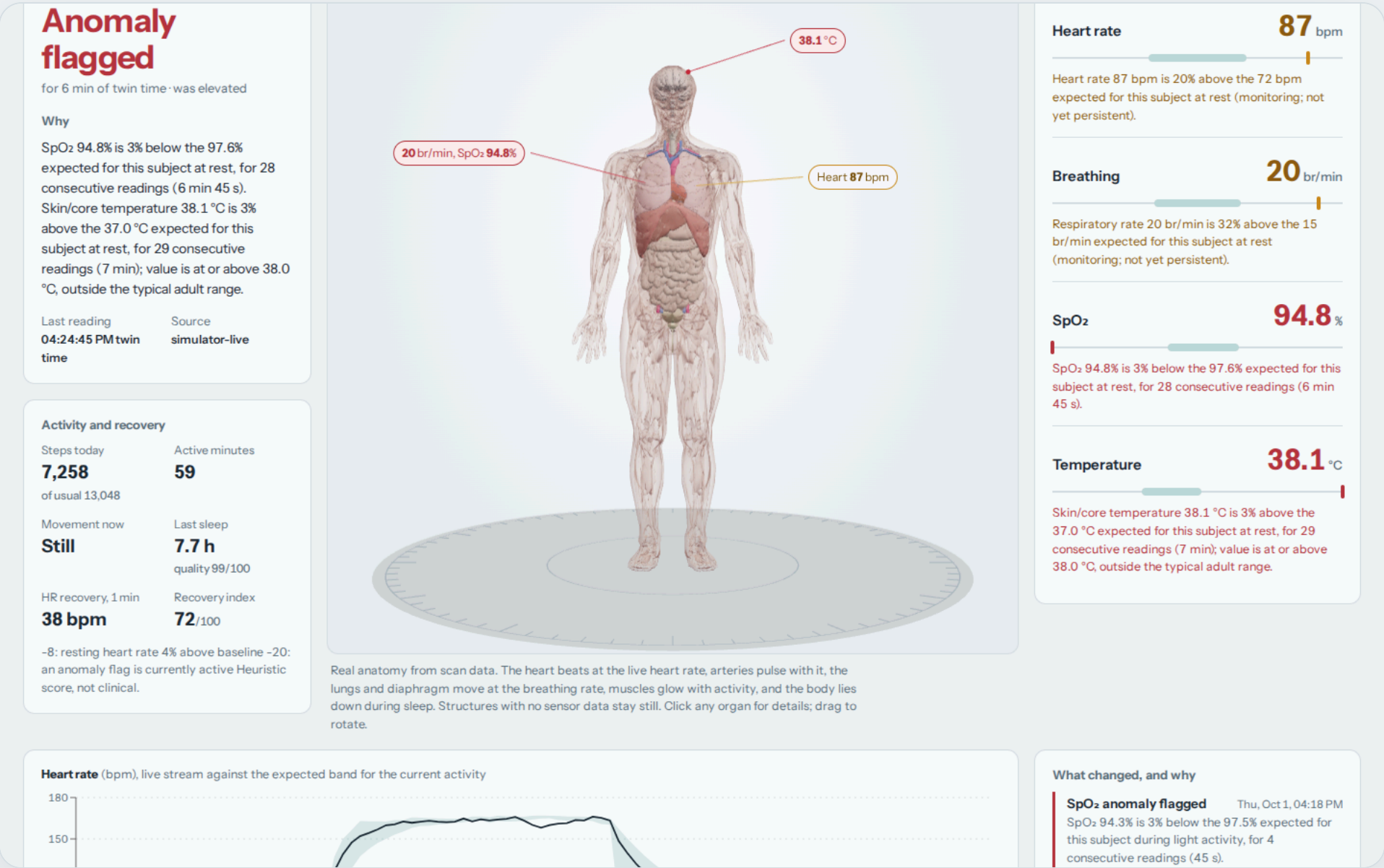
BodyParts3D scan data (CC BY-SA 2.1 JP), 19 structures, built in Blender



Flags that explain themselves

“Heart rate 88 bpm is 24% above the 71 bpm expected for this subject at rest, for 4 consecutive readings.”

- Personal baseline: median and MAD
- Expected band follows activity
- Flag after 4 readings, clear after 3



EVIDENCE

Measured, not claimed

Personal twin

77%

episodes caught

0.0

false alarms per day

Population rule

43%

episodes caught

4.6

false alarms per day

Episode strength	Twin	Rule
Subtle	17%	0%
Moderate	82%	9%
Strong	100%	92%

35 unseen synthetic days, 30 injected episodes. Validated against our own simulator, not clinically. Reproduce: `python -m digital_twin.evaluation`

Ask the twin what-if

9.0 5.5
min to settle min, normal night

Human Health Digital Twin · 08

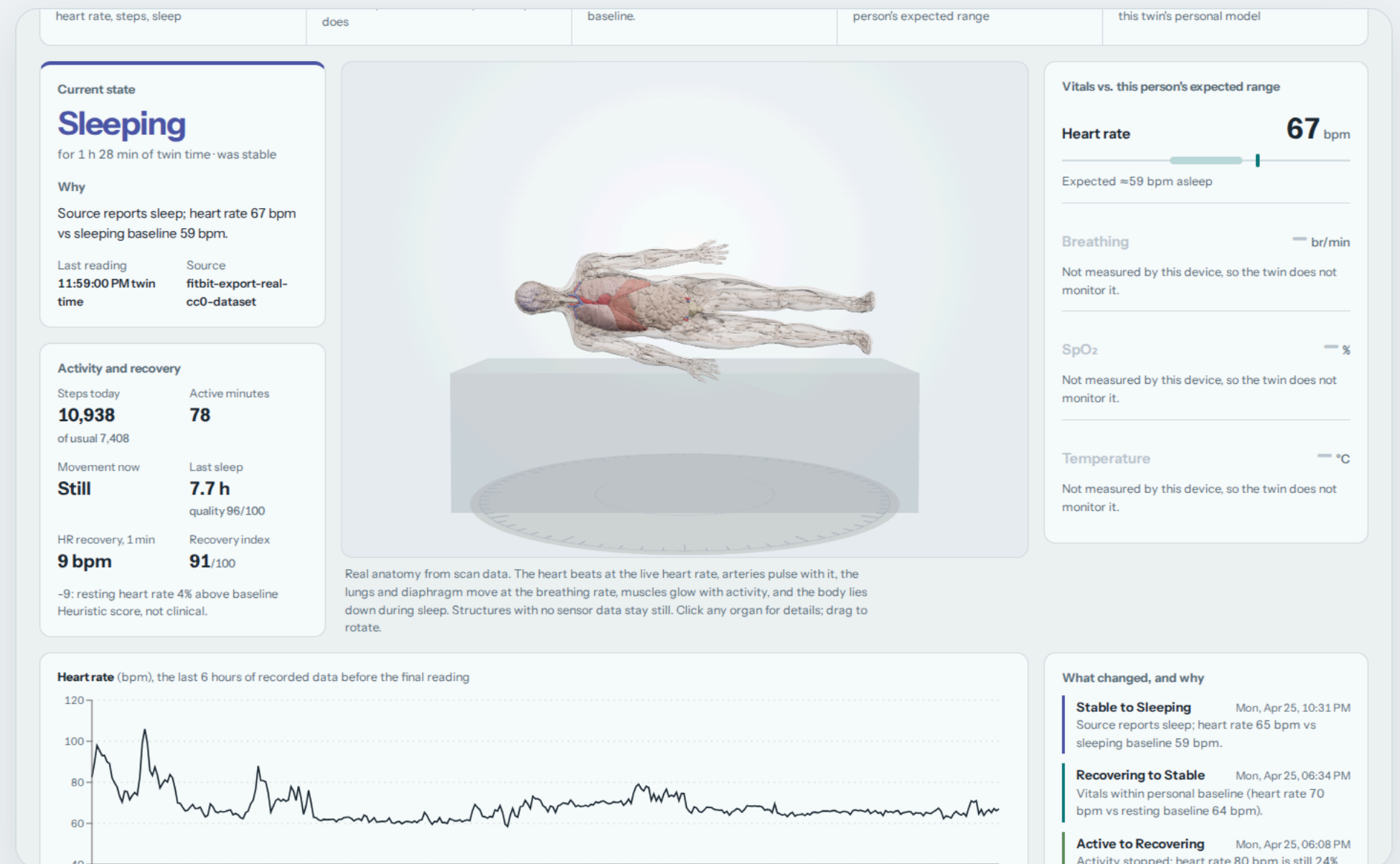


REAL DATA

Works on real people's data

- Import Fitbit, Samsung Health, Google Fit or any CSV
- Bundled: 14 days of a real Fitbit user (CC0)
- Learned resting HR 65, sleep 6.1 h
- Unmeasured vitals shown as “not measured”

Furberg et al. 2016, Fitbit Fitness Tracker Data,
doi:10.5281/zenodo.53894



Your Android phone, connected

- 1

Scan the QR code

Single-use 6-digit code, valid 10 minutes
- 2

Allow Health Connect

Heart rate, steps, SpO₂, breathing, temperature, sleep
- 3

First sync: 7 days

The twin learns this person's baseline
- 4

Then every minute

New readings update the twin live

Connect your Android phone

The **Health Twin** app reads Android **Health Connect**: steps, heart rate, SpO₂, breathing rate, temperature and sleep from your phone and any watch that syncs to it (Samsung Health, Fitbit, Google Fit, Pixel Watch, Mi Fitness...). It sends them to this computer over your Wi-Fi, and the twin updates live.



300 405

Expires in 9:59

Server address for the app

http://192.0.2.2:8011

Waiting for the phone...

Cancel

1. Phone and computer on the **same Wi-Fi**. On Windows, allow Python through the firewall when asked (Private networks).

2. Install the **Health Twin** app (Android 8+, with Health Connect). Get the APK from the GitHub build (Actions → Android app → artifact) or build it in Android Studio; see docs/mobile.md.

3. In the app tap **Scan QR** (or type the server address and the 6-digit code), then **Allow** the Health Connect permissions.

4. Tap **Sync now**. While the app is open it syncs every minute; this page and the twin update by themselves.

Phone	Last sync	Status	Measured	
Fake phone (simulator)	18 s ago	Synced · +1 min	heart rate, spo2, temperature, respiratory rate	Open Disconnect
Fake phone (simulator)	1 min ago	Synced · +1 min	heart rate, spo2, temperature, respiratory rate	Open Disconnect

Health Connect already gathers Samsung, Fitbit, Pixel and Mi watch data, so one Kotlin app covers most watches. Only a hash of each device token is stored; disconnect any phone from the website.

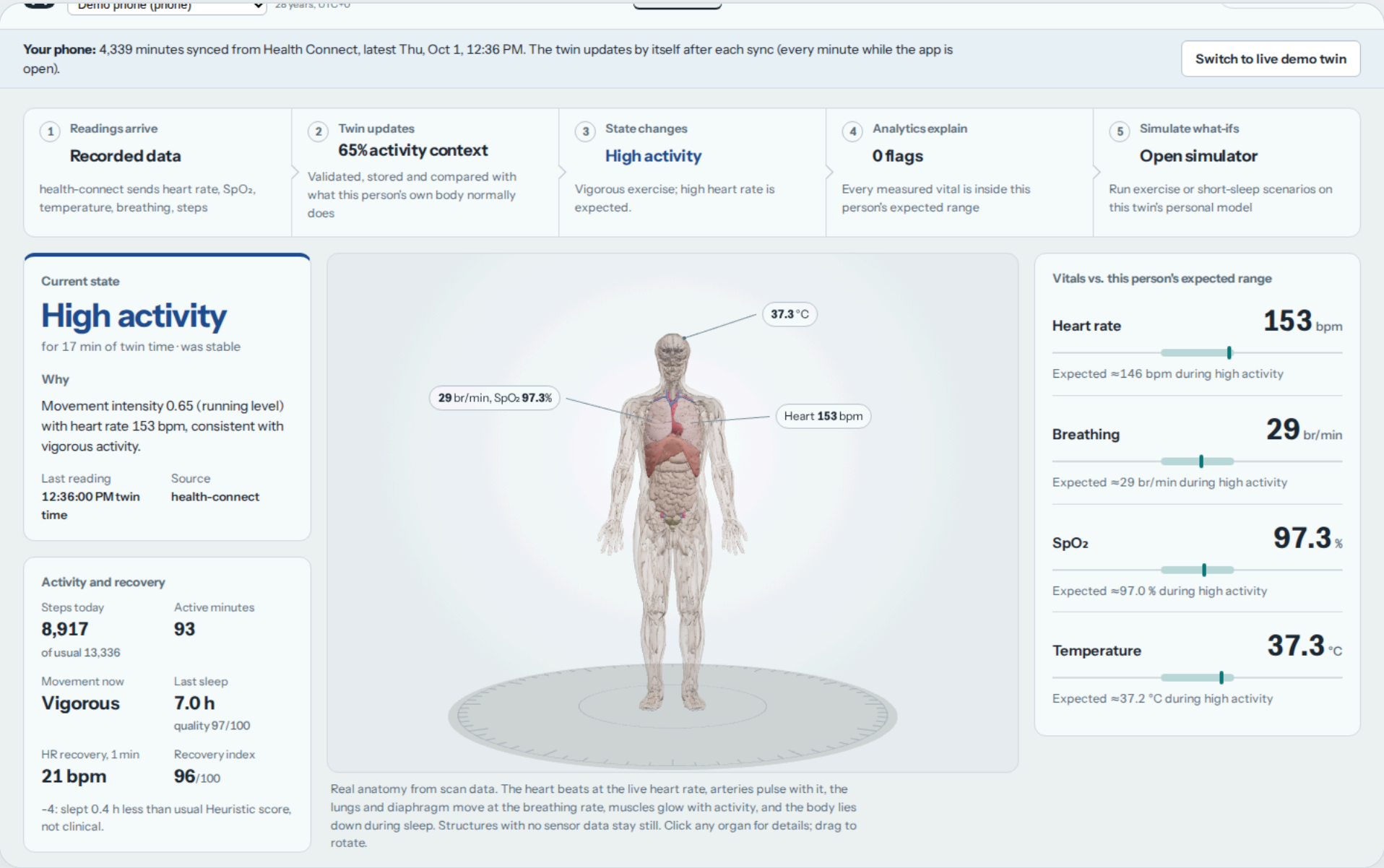
Human Health Digital Twin · 10

LIVE DEVICE DATA

The twin follows the phone

Running at 153 bpm: state **High activity**, every vital inside this person's expected band, **no false alarm**.

Shown: phone sync driven by our test script, which uses the app's exact API and data shape.



Built to be checked

Layer	Technology	Why
Engine	Python, NumPy, pandas	Pure and unit-tested in isolation
API	FastAPI, Pydantic, SQLAlchemy	Typed validation, OpenAPI docs
Web	React, TypeScript, Vite	Live dashboard over WebSocket
3D	Three.js, Blender, BodyParts3D	Real anatomy, data-bound
Mobile	Kotlin, Health Connect	Phone and watches, one app
Quality	70 backend + 7 frontend tests	CI on GitHub Actions, seeded data

OUTCOMES

What we delivered

77%

anomaly episodes caught, 0.0
false alarms per day

4

data paths: simulator, file
exports, Android phone, REST

19

anatomical structures driven
by live data

72 + 7

automated tests, run on every
push

- A working twin that learns a personal baseline and explains every flag
- What-if simulation on the person's own model
- Runs on real Fitbit data and live phone data via Health Connect
- One-click local start, free cloud deployment, open source (MIT)

Limits and what comes next

Limits today

- Accuracy validated on synthetic data only
- Activity estimated from steps
- Android only; syncs while the app is open
- Simplified physiology
- Subtle changes: 17% caught

Roadmap

- Pilot with consenting users
- Learn each person's time constants
- iPhone (HealthKit) and background sync
- BLE and ESP32 sensors
- Accounts, TLS, consent (DPDP Act)

A twin that knows your normal.

It says what changed and why, and lets you ask what-if.

Live demo next · Research prototype, not a medical device · [Team name] · [Member names]